

Java - Debugging Test

Student No.	Reg. No	Student Name (fill in)	Program 1	Program 2	Program 3
1	192311127	GURUPASALA MADHU	1	2	3
2	192311128	PATAKOTTU VENKATA PRANAVA SESHA SAI	4	5	6
3	192311140	Jutooru.Goutham kumar reddy	7	8	9
4	192311190	ENAMALA GOWTHAM	10	1	2
5	192311213	KUNALAREKANTH	3	4	5
6	192311214	NANDI HEMANTH KUMAR	6	7	8
7	192311271	DINNEPATI SINDHU PRASAD	9	10	1
8	192311312	JAINAB MEKALA	2	3	4
9	192311362	GIRIJALA VAMSI KRISHNA	5	6	7
10	192311381	MUCHELI MOHITH GUPTA	8	9	10
11	192321052	TANUJA P	1	2	3
12	192324090	SOORIYAVEL. B	4	5	6
13	192324092	SABARISH R	7	8	9
14	192324105	SHAM SUNDER M	10	1	2
15	192324139	SIRIYALA MUBINA	3	4	5
16	192324162	METTIMI VAISHNAVI	6	7	8
17	192324182	BOMMU LAKSHMI LOHITHA	9	10	1
18	192324219	KETHA DEVI VARA PRASAD	2	3	4
19	192324221	SANKA MOHAN KRISHNA BALAJI	5	6	7
20	192324252	KADIYALA DEDEEPPYA	8	9	10
21	192324284	VAIDYAM PRASAD SNEHA	1	2	3
22	192324290	D GOPI	4	5	6
23	192324306	CHODAM NISHA	7	8	9
24	192324307	CHENNURU THARUNIKA REDDY	10	1	2
25	192325041	MORASU VAMSI KRISHNA	3	4	5
26	192325124	SAI NIKHIL O	6	7	8
27	192372043	CHENNAIAH GARI TEJASHWI	9	10	1
28	192372056	HASTHAVARAM SANJAY REDDY	2	3	4
29	192372138	LEKKALA VISHNU VARDHAN REDDY	5	6	7
30	192372174	MAREDUKONDA PRAMITH	8	9	10
31	192372210	YANUGONDA. SAI MOHITH	1	2	3
32	192372270	MAHENDRAKAR SAI CHANDRA	4	5	6
33	192372302	TUMMALA SRISHANTH	7	8	9
34	192372369	POLIREDDY REVANTH REDDY	10	1	2
35	192372373	METTIREDDY MOKSHITH REDDY	3	4	5
36	192472217	SHAIK ISSAVALI	6	7	8
37	192472322	MOHAMMAD AKRAM K	9	10	1

Program 1: OOP Basics - Class, Object, Encapsulation, Constructor

// Topic: Basics of OOP Concepts

// Instructions: This program has 3 bugs. Find and fix them so the output is correct.

```
class Student {  
    private String name;  
    private int age;  
    public Student(String name, int age) {  
        name = name;  
        this.age = age;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setAge(int age) {  
        age = age;  
    }  
    public void display() {  
        System.out.println("Name: " + name + ", Age: " age);  
    }  
}  
public class Program1_OOPBasics {  
    public static void main(String[] args) {  
        Student s1 = new Student("Arun", 20);  
        s1.setAge(21);  
        s1.display();  
    }  
}
```

/*

Expected Output:

Name: Arun, Age: 21

*/

Program 2: Method Overloading

// Topic: Overloading

// Instructions: This program has 3 bugs. Find and fix them so the output is correct.

```
class Calculator {  
    int multiply(int a, int b) {  
        return a * b;  
    }  
    int multiply(int a, int b) {  
        return a * b * 2;  
    }  
    long multiply(long a, long b) {  
        return a * b;  
    }  
    float multiply(float a, float b) {  
        return a * b;  
    }  
}  
public class Program2_Overloading {  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println(calc.multiply(2, 3, 4));  
        int result = calc.multiply(2.5f, 3.5f);  
        System.out.println(result);  
    }  
}
```

/*

Expected Output (after fixing):

6

8.75

*/

Program 3: Method Overriding

// Topic: Overriding

// Instructions: This program has 3 bugs. Find and fix them so the output is correct.

```
class Animal {
    public String makeSound() {
        return "Animal makes a sound";
    }
    public void eat(String food) {
        System.out.println("Animal eats " + food);
    }
    public String getInfo() {
        return "Generic Animal";
    }
}
class Dog extends Animal {
    @Override
    private String makeSound() {
        return "Dog barks";
    }

    public void eat(int food) {
        System.out.println("Dog eats " + food);
    }
    @Override
    public int getInfo() {
        return 1;
    }
}
public class Program3_Overriding {
    public static void main(String[] args) {
        Animal a = new Dog();
        System.out.println(a.makeSound());
        a.eat("Bone");
        System.out.println(a.getInfo());
    }
}
/*
```

Expected Output:

Dog barks

Dog eats Bone

Dog Info

*/

Program 4: Packages (FILE 1 of 2 — save this as Rectangle.java inside a folder named "shapes")

// Topic: Package

// Instructions: This program (across both files) has 3 bugs. Find and fix them so the program compiles and runs correctly.

```
import java.lang.Math;
package shape;
class Rectangle {
    double length;
    double width;
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    double getArea() {
        return length * width;
    }
}
package app;
import shapes.Rectangle;
public class Program4_Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(5.0, 3.0);
        System.out.println("Area: " + r.getArea());
    }
}
/*
```

Folder structure expected:

shapes/Rectangle.java (package shapes;)

app/Program4_Main.java (package app;)

Expected Output (after fixing):

Area: 15.0

*/

Program 5: Inheritance

// Topic: Inheritance

// Instructions: This program has 3 bugs. Find and fix them so the output is correct.

```
class Vehicle {
    private String brand = "Generic";
    protected int speed = 0;
    public Vehicle() {
        System.out.println("Vehicle created");
    }
    public void displayBrand() {
        System.out.println("Brand: " + brand);
    }
}
class Engine {
    protected String engineType = "Petrol";
}
class Car extends Vehicle, Engine {
    public Car(int speed) {
        this.speed = speed;
        super();
    }
    public void showDetails() {
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed);
    }
}
public class Program5_Inheritance {
    public static void main(String[] args) {
        Car c = new Car(120);
        c.showDetails();
    }
}
/*
Expected Output:
Vehicle created
Brand: Generic
Speed: 120
*/
```

Program 6: Static Keyword

// Topic: Static

// Instructions: This program has 3 bugs. Find and fix them so the output is correct.

```
class Counter {
    static int count = 0;
    int id;
    public Counter() {
        count++;
        id = count;
    }
    public static void showCount() {
        System.out.println("Total Counters: " + count);
        System.out.println("Current id: " + id);
    }
    public void sayHello() {
        System.out.println("Hello from Counter id " + id);
    }
    public static void greet() {
        sayHello();
    }
}

public class Program6_Static {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter.showCount();
        int total = Counter.count;
        System.out.println("Total: " + Program6_Static.total);
    }
}

/*
Expected Output:
Total Counters: 2
Total: 2
*/
```

Program 7: Final Keyword

// Topic: Final

// Instructions: This program has 3 bugs. Find and fix them so the output is correct.

```
final class Constants {
    static final int MAX_SPEED = 120;
}

class SuperClass {
    public final void show() {
        System.out.println("This is a final method");
    }
}

class SubClass extends SuperClass {
    @Override
    public void show() {
        System.out.println("Trying to override");
    }
}

class AnotherClass extends Constants {
}

public class Program7_Final {
    public static void main(String[] args) {
        final int speed;
        speed = 100;
        speed = 150;
        System.out.println(speed);
        System.out.println(Constants.MAX_SPEED);
    }
}

/*
Expected Output:
100
120
*/
```

Program 8: Exception Handling

// Topic: Exception Handling

// Instructions: This program has 3 bugs. Find and fix them so the output is correct.

```
import java.io.IOException;

class Processor {
    void readFile() throws IOException {
        throw new IOException("File not found");
    }

    void process() {
        readFile();
    }
}

public class Program8_Exception {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            int result = a / b;
            System.out.println(result);
        } catch (Exception e) {
            System.out.println("General error: " + e);
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic error: " + e);
        }
    }

    int[] arr = {1, 2, 3};
    System.out.println(arr[5]);
    System.out.println("Done");
}

/*
Expected Output:
Arithmetic error: java.lang.ArithmeticException: / by zero
Array index error: java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds
for length 3
Done
*/
```

Program 9: Multithreading

// Topic: Thread

// Instructions: This program has 3 bugs. Find and fix them so the output is correct.

```
class MyThread extends Thread {  
    public void run(int x) {  
        System.out.println("Thread running with x = " + x);  
    }  
}
```

```
class Counter implements Runnable {  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
}
```

```
public class Program9_Thread {  
    public static void main(String[] args) throws InterruptedException {  
        MyThread t1 = new MyThread();  
        t1.run();  
        Thread.sleep(100);  
        System.out.println("Main thread finished");  
    }  
}
```

/*

Expected Output (order of first two lines may vary slightly, but t1 should run on a separate thread):

Thread running

Main thread finished

*/

Program 10: Comprehensive (Interface, Abstract Class, Inheritance, Static, Override, Exception)

// Topic: Mixed / Capstone

// Instructions: This program has 4 bugs spanning multiple concepts. Find and fix them so the output is correct.

```
interface Shape {
    double getArea();
}

abstract class AbstractShape implements Shape {
    protected static int shapeCount = 0;
    protected final String shapeName;

    public AbstractShape(String name) {
        shapeName = name;
        shapeCount++;
    }

    public abstract double getArea();

    public void displayInfo() {
        System.out.println(shapeName + " Area: " + getArea());
    }
}

class Circle extends AbstractShape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getArea(int unused) {
        return Math.PI * radius * radius;
    }
}

class ShapeProcessor {
    void process(Shape s) {
        try {
            System.out.println("Processing area: " + s.getArea());
        } catch (Exception e) {
```

```
        System.out.println(e);
    } finally {
        return;
    }
}

public class Program10_Comprehensive {
    public static void main(String[] args) {
        Circle c = new Circle(5.0);
        c.displayInfo();
        System.out.println("Total shapes: " + AbstractShape.shapeCount);
    }
}

/*
Expected Output:
Circle Area: 78.53981633974483
Total shapes: 1
*/
```